

ML and Metrics Final

Jack Ogle and Izzy Allum

May 2022

Introduction

Question

The question this report seeks to answer is what makes music popular? Specifically, can we predict what songs will become hits, or what songs individuals will enjoy? There are many different components that make up a song, like the artist, genre, lyrics, instrumentals, tempo, and more. But because there are so many potential explanatory variables, it is difficult to easily discern exactly why certain songs are more popular than others.

From a commercial perspective, understanding which songs will become hits or will be liked by certain individuals is significant. For example, a large music streaming company could improve its advertising models, and thus its profits, with this knowledge. If they can predict which songs will become popular and guarantee more impressions for ads played around those songs, they can raise prices for those ad slots. As for the ability to predict which songs individual users would enjoy, if streaming companies can provide spot-on recommendations to their users, users would be more likely to continue using the app, or even recommend the platform to their friends. This increased platform usage would result in increased profit for the company.

Literature Review

Many prior studies have been conducted on the prediction of popular songs and on music and other entertainment recommendation systems.

One previous paper by Araujo et. al tracked Spotify's Top 50 and Viral 50 playlists and downloaded their songs' data with the Spotify API for about a year. With these playlists, they determined "popular" and "unpopular" songs and produced algorithms using Gaussian Naive Bayes, k-nearest neighbors, logistic regression, and SVM methods to predict popular songs. They found the most accurate method was SVM.

Another paper by Giusti similarly used data from the Spotify Global Top 50 chart. They used explanatory variables from the Spotify API, well as audio analysis of 30 second clips from the songs. The researchers used Random Forests, Bernoulli and Gaussian Naive Bayes, and SVM methods, and managed to produce a model that predicted, with 89.09% accuracy, whether a song would be popular two months later.

Zhao and Li wrote a paper in 2018 about the music recommendation system based on Hadoop. They used a k-nearest neighbors algorithm to grouped similar users together, and they recommended songs like by similar users to individuals looking for new music. They found that using k-nearest neighbors greatly improved on the accuracy of the previous recommendation system, and that when K equalled 4, they had the greatest success with their recommendations.

Akshay et al. also used k-means clustering, but they did so with the "Million Songs Dataset", and they clustered songs rather than users.

Two other papers by Zhang et al. and Nair et al. attempted song and movie recommendations, respectively, with random forest methods, and Nair found that random forests had a lower root means squared error than k-means clustering.

Data

For this paper we used the Hit Predictor Dataset found on Kaggle. The dataset combines Spotify and Billboard API data for decades of song releases, but we used just the most recent decade's data due to relevancy concerns. The data contains variables for the name, artist, uri, danceability, energy, key, loudness, mode, speechiness, acousticness, instrumentalness, valence, tempo, duration, time signature, time before the chorus, and the number of sections of a song, as well as a binary variable for whether it was a "hit" or a "flop".

A song is considered a hit if it is included in Billboard's weekly list of Hot-100 tracks from 2010 to 2019. For a song to be included as a flop in the data set, it must have never made it on the Hot-100 tracks list that decade, the song's artist must not have appeared on the list either, the song's genre must be non-mainstream (non-pop), and the genre must not have a song in the list. The flop songs must still be made for the US market though.

Methodology

Our first goal was to divide the data into a training subset and a testing subset, and using a machine learning algorithm training on the training data, predict which song would be a "hit" in the testing set. This is slightly different from the aforementioned literature that looks at music as it reaches the current charts, as we are looking at data from the past decade (2010-2019). Although this means that we are looking at less contemporary, and perhaps less relevant, data, we have a broader view of the data and its trends. Therefore, we can predict songs become popular following a longer period of time after their release, in comparison to, for example, the Rauho et. al. paper that only looks 2 months ahead. Additionally, the prior papers look at songs that are already in the top charts or viral charts and that are already popular to some extent. Our data allows for better comparison between popular and unpopular songs as about half of the songs are not anywhere near these charts.

To predict which songs would be popular, we chose to use a random forest as it seemed a reasonable choice based on our data. Generally speaking, random forests are broadly applicable and fairly accurate. While they struggle extrapolating outside the range of the training data, that is not an issue for us as our training and test data are similar (we are predicting and training with data from the same decade, and we figured that music trends would be consistent enough for a random forest). Additionally, random forests do not assume normality of the data, which is suitable for our data, as many of the variables are not normally distributed. Lastly, while random forests are a bit more of a black box, meaning it is not so clear what the exact relationship between the explanatory variables and the outcome variable is, this is not of great concern for our principal question. If the main goal of a streaming company is to predict what song will be popular for the sake of increasing profit, they'd be happy to trade this understanding for an increase in accuracy. So, as long as the random forest proves to be useful, we should not be bothered that it is harder to interpret the process of prediction.

In addition to the random forest, we also trained a logistic regression and used it to predict the popularity of the songs in the test data. A logistic regression seemed the simplest method of predicting a binary outcome, so we chose that as our baseline with which to compare the efficacy of the random forest.

Our second goal was to build a recommendation system using machine learning methods. Our baseline method was k-means clustering as we saw that it was successfully used in many other papers. We also used matrix completion as another approach.

In order to make this recommendation system as robust as possible, we used an approach that optimizes both time and accuracy. Given that the aim of our project is, in part, to provide users timely and relevant song recommendations, our model must reflect that. We have done a comprehensive literature review on recommendation systems. They often employ an approach with multiple heuristics. We have chosen to do K-mean clustering and matrix completion.

Our proposal for a song recommendation system would be to use k-means clustering for new users without any user history and for those who do not use the rating or liking features. For these users, it would be hard to use matrix completion as generating a predicted matrix is difficult with such low-rank user data. However, once a user provides information by interacting with the streaming platform and its songs, we'd move onto using matrix completion instead.

K-means is an unsupervised machine learning algorithm that clusters the data into groups. Supervised machine learning encompasses the first two methods that we implemented in our prediction system. These methods are classification (Random Forest) and regression (Logistic Regression). The major difference between unsupervised and supervised is that unsupervised does not rely on the labels of our data. It instead looks for the patterns within the data. For example, in k-means clustering models, we do not need to clearly specify any outcome nor explanatory variables. In logistic regression, this is not the case because when we define the model we clearly define an outcome and some explanatory variables. With the supervised model, we attempt to predict causal relationships. With unsupervised learning, we are investigating the structure of the data when the structure of that data is non-obvious.

The k-means algorithm clusters the data into k separate clusters, or groups, with equal variance while maintaining a minimum of inertia. Inertia is a multivariate measure of the amount of variation in our data. More specifically, it is the sum-of-squares within each cluster. Given observations, k-means will attempt to generate k clusters with the least variance possible over a set of iterations (S). For this algorithm, we need to specify the exact number of clusters we want. We will estimate the average silhouette score for each number of clusters. The higher the silhouette score, the better the model, This is because it accounts for the mean distance between a sample and all other points in the same class and the mean distance between a sample and all other points in the next nearest cluster.

$$\arg \min_{\mathbf{S}} \sum_{i=1}^k \sum_{\mathbf{x} \in S_i} \|\mathbf{x} - \boldsymbol{\mu}_i\|^2$$

Essentially, the k-means method will generate groups of similar songs and recommend that the user listen to this cluster. Therefore, it will be the first line of recommendation. Based on the current song the user is listening to, it will recommend them a song or playlist from that same cluster.

Once a user has liked and rated a significant number of songs, our system will employ Matrix Completion. At the highest level, Matrix Completion is taking a sparsely populated matrix and generating the missing values with statistical engineering. The Matrix Completion framework has a few variants as seen in (Johnson, 1990). The data from each individual user will be low rank because the number of linearly independent rows and columns is smaller than the total number of rows and columns. This is because each user

will only rate or listen to a very small subset of the total number of songs available on the streaming platform.

The most basic Low Rank model is to solve the convex problem of minimizing the nuclear norm of a predicted Matrix X such that X is equal to the observed Matrix M .

$$\text{minimize } \|X\|_* \quad (1)$$

$$\text{subject to } P_\Omega(X) = P_\Omega(M)$$

where:

$$P_\Omega(X)_{ij} = \begin{cases} X_{ij}, & (i, j) \in \Omega \\ 0, & \text{otherwise.} \end{cases}$$

We will then compare that results of that method to the results of this convex problem:

$$\begin{aligned} &\text{minimize } \|X\|_* \quad (2) \\ &\text{subject to } \sum_{(i,j) \in \Omega} (M_{ij} - X_{ij})^2 \leq \delta \end{aligned}$$

We chose these two matrix completion models because we think they will fit our data. More specifically (1) was chosen because, according to our literature review, it is the benchmark for matrix completion (Candes, Recht). We cannot minimize the rank because the algorithms required to solve it take too long. We need an efficient recommendation model to quickly recommend songs to users in real time. If we minimize nuclear norm, which is the sum of the singular values over the constraint set, our model will be much faster. The second model (2) also minimizes the nuclear norm. However the constraint is less rigid. It is simply measuring ensuring that sum of the squared residuals between the observed matrix and the predicted matrix stays below a certain threshold. The hope is that we might be able to recover the matrix faster and still maintain the accuracy of our first model. Our goal is to accurately complete in a sparse matrix. Our main measure of this is the Mean Squared Error. The solution with the lowest mean squared error is the best estimate. This is because the mean squared error calculates the difference between the estimator and the real.

Results and Discussion

Logistic Regression

First, we ran a logistic regression as our baseline comparison in predicting songs' popularity. Before running the regression, we normalized the applicable data so all the variables would be within similar ranges. Then I regressed the dummy variable for being a "hit" on danceability, energy, key, mode (major or minor scale), speechiness, acousticness, instrumentalness, liveness, valence, tempo, duration (in minutes), time signature, what time chorus hit, number of sections in the song, and if there is a featured artist.

We chose not to include the track title or the artist in the regression. We did not include the track title because I thought they would only be useful if we were able to do linguistic analysis on them, perhaps determining the sentiment of or the emotions invoked by the words in the title. However, the packages we know for word analysis did not include words for every song title, so including title linguistic analysis would have meant lots of empty observations. For this reason, we left song titles out of our analysis. As for the artist variable, we didn't want to over fit to the artist variable.

Running this first regression, there were five variables that did not have statistically significant coefficients: key, liveness, valence, what time the chorus hit, and the number of sections. Seeing this, we ran a second regression removing the five non-statistically significant coefficients but keeping the rest. Having

done so, the coefficients all remained significant, most with confidence level of $\alpha < 0.1$.

Having trained our logistic regression on the training data, we ran it on our testing data. The logistic regression predictions for the testing data were given in the form of percentage odds that the song was a hit, so we set the predictions as 1 if the percentage was greater than or equal to 50% and 0 otherwise.

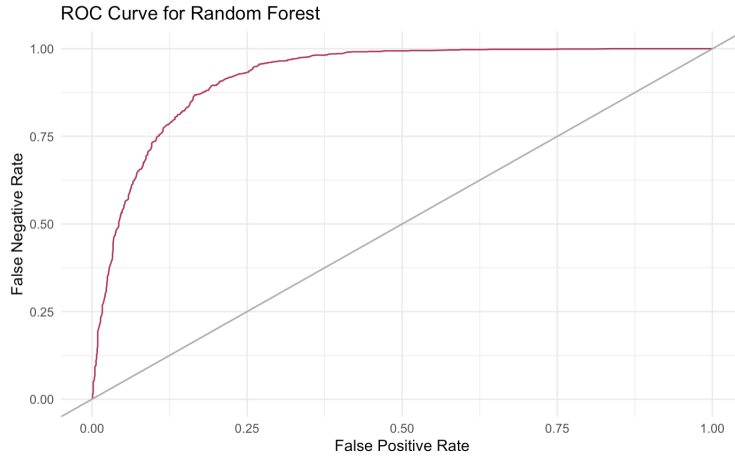
This prediction method was fairly successful with an approximately 80.93% success rate, with a 95% confidence interval of [0.7924, 0.8254]. If you break this success rate into accurate hit and flop predictions, 85.61% of hits were correctly predicted, and 76.01% of flops were correctly predicted. You can see this difference reflected in the false positive and negative rates. The false negative rate was about 7.37%, and the false positive rate was about 11.71%, meaning the logistic regression was about 1.5 times as likely to predict a flop would be a hit than to say a hit would be a flop.

Looking at the coefficients in the regression, the four coefficients of the largest magnitude were energy, loudness, whether there was a featured artist, and instrumentality, in ascending order. Instrumentality and energy had negative coefficients, and loudness and featured artist had positive coefficients, suggesting that the more energetic and instrumental a song was, the less likely it was to be a hit, and that loud songs with featured artists were more likely to be hits. While it might seem a little counter-intuitive that instrumentals and energy are both negative coefficients while approximately opposite characteristics, songs with heavy focus on instrumental and little vocals, like classical compositions, are not the popular in mainstream music, and neither are the most extremely energetic songs, like death metal. Most pop songs are somewhere in the middle, so it makes sense that the extremes are punished when predicted popularity. Pop songs do tend to be loud, so it makes sense that loud songs are likely to be popular. And songs with features are often collaborations between two well-known artists, meaning there are two large fan bases supporting the song, so it also follows that songs with features are also more likely to be popular.

Random Forest

Next, we ran our machine learning method of interest: the random forest. We ran it on the same training subset as the logistic regression, and we used the same variables in our initial random forest as in our initial logistic regression. We then looked at the variable importance of the different explanatory variables, and removed those that had little effect on the accuracy of the random forest output. We repeated this process once more, until there were no variables of low importance.

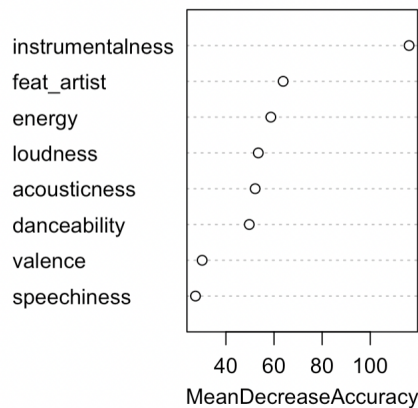
Next, we used the random forest to predict whether songs in the test set were hits or flops, and the results of this prediction were accurate at a rate of about 85.13%. While it is just a slightly higher rate than that of the logistic regression, the confidence interval span was [0.8359, 0.8658], meaning it does not overlap with the confidence interval of the logistic regression. So the difference in accuracy is statistically significant. Looking at the ROC curve for the random forest, it is a fairly strong predictor.



86.84% of hits were accurately predicted and 83.33% of the flops were accurately predicted. In comparison to the logistic regression, the difference in the accuracy of hits and flops predictions closed substantially. The false negative rate was 6.74%, and the false positive rate was 8.13%. Both were lower with the random forest than they were with the logistic regression, although the false positive rate was still higher than the false negative rate.

	Overall Accuracy (%)	Accuracy of Hits (%)	Accuracy of Flops (%)	False Positive Rate (%)	False Negative Rate (%)
Logistic Regression	80.93 [0.7924, 0.8254]	85.61	76.01	11.71	7.37
Random Forest	85.13 [0.8359, 0.8658]	86.84	83.33	8.13	6.74

Similar to the logistic regression, the instrumentality, the presence of a featured artist, energy, and loudness were the most important variables, in descending order. In both the random forest and the linear regression, instrumentality was the most important variable, but the degree to which varies between methods. With the logistic regression, the coefficients for featured artist, loudness, and instrumentality were all very close. But with the random forest, instrumentality was the most important by a wide margin.



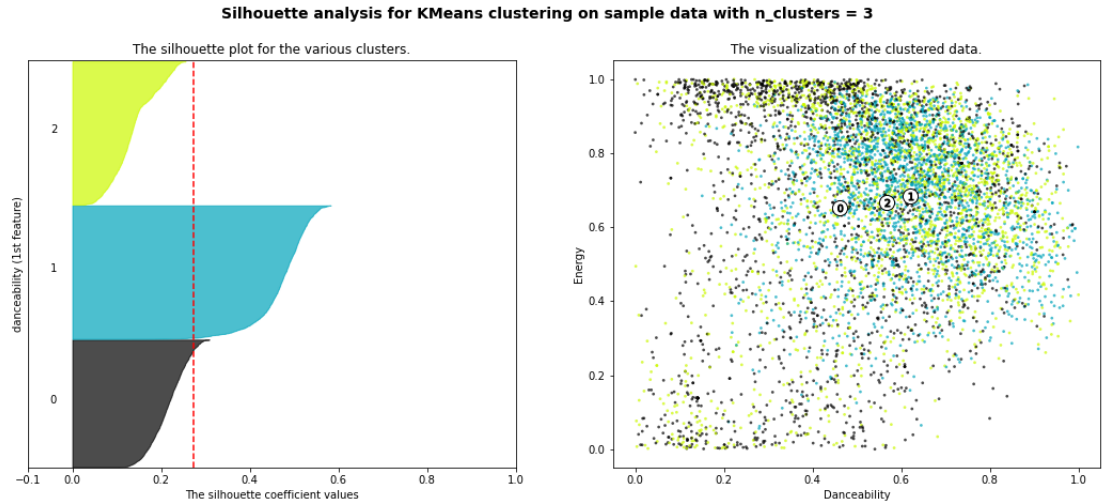
K-Means Clustering

When formulating our model, we opted to include a silhouette score in order to estimate exactly how many clusters we should choose. Because the silhouette score is the highest for $n = 5$ the most optimal number of clusters is 5. The silhouette score is calculated with the (a) mean distance between a sample and all other points in the same class and the mean distance between a sample and (b) all other points in the next nearest cluster. A score closer to zero means that the clusters are overlapping. And a score closer to 1 means that the clusters are distinct and not touching.

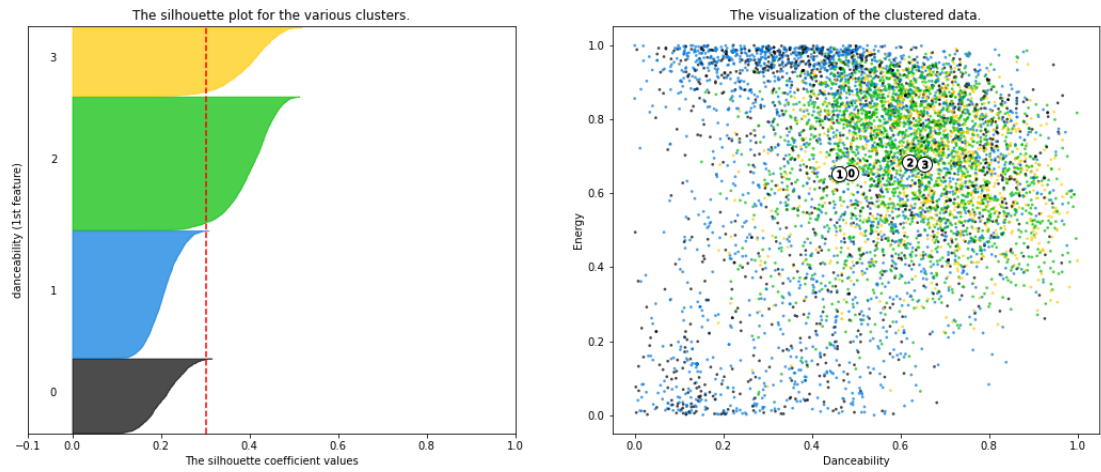
$$s = \frac{b - a}{\max(a, b)}$$

n Clusters	Mean Silhouette Score
3	0.27373
4	0.30330
5	0.33143
6	0.31697
7	0.25956

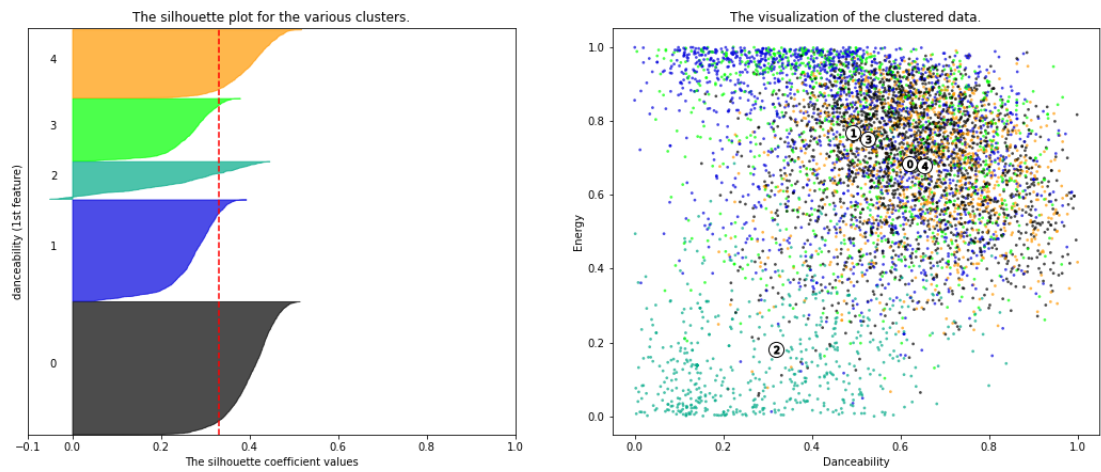
We opted not to plot all 16 of the dimensions to visualize each cluster. We are just comparing the 1st feature and the 2nd feature. Which is danceability and energy. We did this for simplicity and the other figures that contain the other features of our data look similar. Visualizing all the possible features at once would be confusing and visualizing all the possible combinations would be redundant.



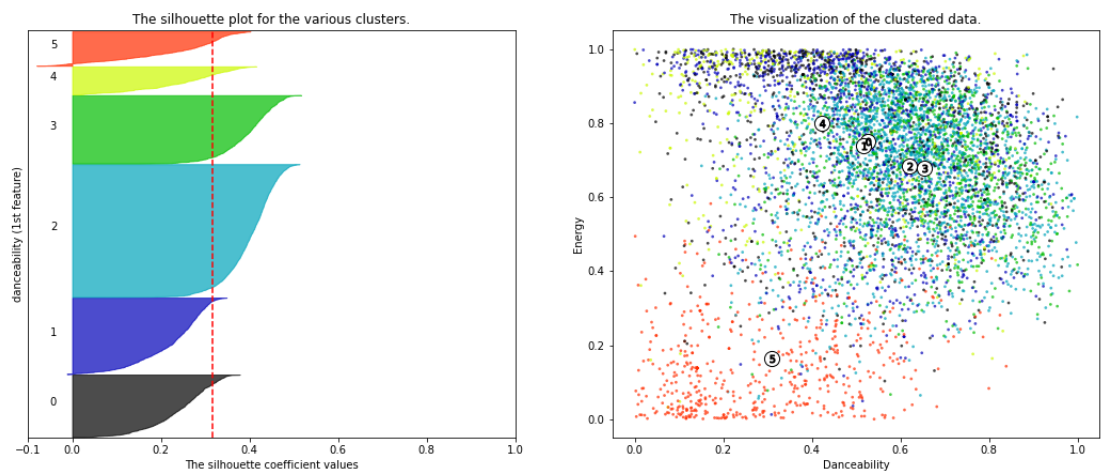
Silhouette analysis for KMeans clustering on sample data with $n_clusters = 4$

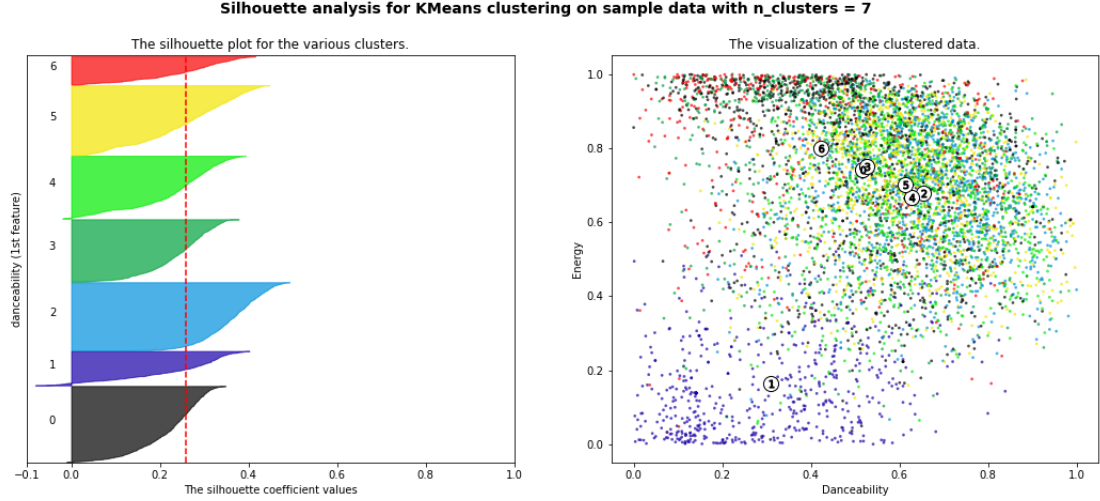


Silhouette analysis for KMeans clustering on sample data with $n_clusters = 5$



Silhouette analysis for KMeans clustering on sample data with $n_clusters = 6$





Matrix Completion

We were only able to calculate the mean squared error (50.34) and the rank for one of the models, specifically the matrix completion model that minimizes the nuclear norm of the predicted matrix such that the omega indices of the predicted matrix is equal to the same indices of the observed matrix. As we were only able to generate one resultant model, so we are not able to draw a conclusion. Our goal was to generate at least two models for matrix completion and compare the results for both the rank and the Mean Squared Error. The model with the lower MSE would be the prevailing model. Without the ability to make this comparison we are not able use to the matrix completion model in our recommendation system.

Summary and Conclusion

In this paper, we attempted to develop an algorithm to predict hit songs using machine learning methods and to provide song recommendations. For the hit song predictions, we used logistic regressions and random forests, and for the song recommendations, we used K-mean clustering and matrix completion.

For hit song prediction, the random forest was more successful, but both methods were fairly accurate. The logistic regression and the random forest correctly predicted the "hit" or "flop" status of the test set songs 80.93 and 86.84% of the time, respectively. Interestingly, for both methods, the hits were more accurately predicted than the flops, suggesting hit are in some way more identifiable.

For the song recommendation system, we first found that the ideal number k is equal to 5, as in we cluster all the songs into 5 groups. With this value, the average silhouette score was the highest, suggesting the songs are most similar. Our matrix completion analysis was unfortunately not successful. We attempted to use our personal Spotify listening data joined with the Spotify Hits data to come up with song recommendations for ourselves, but the joined data was too sparse to train the matrix completion sufficiently. This means we were not able to draw any meaningful conclusions from our matrix completion analysis.

While this paper only looks into Spotify rankings, our proposed models could also potentially work on other settings, provided different data and variables, like for movies, tv shows, or any other entertainment industries. As with music streaming companies, there are surely economic rewards for other entertainment companies for accurately predicting popular media and for having a strong recommendation systems.

We identify a few limitations in our research, the largest being limitations imposed by the data we worked with. This is most clear in the sparsity of our data for matrix completion training. The data was sparse because the data for our Spotify listening did not share the same variables as the hits data. Therefore, we could only use the songs in our Spotify listening already in the hits data since we were using the hits data to complete the matrix. If we were able to find new listening data with the same variables as our hits data, and therefore decrease the sparsity, our matrix completion results would be far more successful.

The data also had a very bimodal representation of songs, including only very popular and very unpopular songs. If our hit predictor and song recommendation systems were to be applied to a real-life scenario with more songs that vary along the scale of popularity, we do not know how well they would work with middle-tier popular songs. Therefore, if we were to extend this paper, we would want to find a wider range of songs to train and test with.

References

Soares Araujo, C. V., Pinheiro de Cristo, M. A., amp; Giusti, R. (2020). A model for predicting music popularity on streaming platforms. *Revista De Informática Teórica e Aplicada*, 27(4), 108–117. <https://doi.org/10.22456/2175-2745.107021>

Araujo, C. S., Cristo, M., amp; Giusti, R. (2019). Predicting music popularity on streaming platforms.

Anais Do Simpósio Brasileiro De Computação Musical (SBCM 2019). <https://doi.org/10.5753/sbcm.2019.10436>

Yufeng, Zhao Xinwei, Li. (2018). Design and Implementation of Music Recommendation System Based on Hadoop. *International Journal of Advanced Network, Monitoring and Controls*. 3. 126-132. [10.21307/ijanmc-2018-045](https://doi.org/10.21307/ijanmc-2018-045).

Heng-Ru Zhang, Fan Min, Xu He, "Aggregated Recommendation through Random Forests", *The Scientific World Journal*, vol. 2014, Article ID 649596, 11 pages, 2014. <https://doi.org/10.1155/2014/649596>

Ajesh, A Nair, Jayashree Jijin, P. (2016). A random forest approach for rating-based recommender system. 1293-1297. [10.1109/ICACCI.2016.7732225](https://doi.org/10.1109/ICACCI.2016.7732225).

Akshay, A. S., Madhura, M., Mounika, B., Rohit, H. S., amp; Vikranth, B. M. (n.d.). Music Recommendation System. *International Journal of Latest Technology in Engineering, Management amp; Applied Science*. Retrieved May 31, 2022, from <https://www.ijltemas.in/DigitalLibrary/Vol.8Issue3/84-86.pdf>

Laurent M. (2008) Matrix Completion Problems. In: Floudas C., Pardalos P. (eds) *Encyclopedia of Optimization*. Springer, Boston, MA. https://doi.org/10.1007/978-0-387-74759-0_355

E. J. Candes and Y. Plan, "Matrix Completion With Noise," in *Proceedings of the IEEE*, vol. 98, no. 6, pp. 925-936, June 2010, doi: [10.1109/JPROC.2009.2035722](https://doi.org/10.1109/JPROC.2009.2035722).

E. J. Candes and Y. Plan, "Matrix Completion With Noise," in *Proceedings of the IEEE*, vol. 98, no. 6, pp. 925-936, June 2010, doi: [10.1109/JPROC.2009.2035722](https://doi.org/10.1109/JPROC.2009.2035722).